

System Programming Project 2

담당 교수 : 김영재 교수님

이름 : 이석희

학번 : 20200595

1. 개발 목표

- 해당 프로젝트에서 구현할 내용을 간략히 서술.
- (MyShell을 만드는 전체적인 개요에 대해서 작성하면 됨.)

이 프로젝트의 목표는 사용자가 입력한 명령어를 해석하고, 이를 새로운 프로세스를 통해 실행할 수 있는 간단한 셸(Myshell)을 구현하는 것이다. 특히, 단일 명령 실행을 넘어 복잡한 파이프라인 명령어 처리, 백그라운드 실행, 시그널 핸들링 등을 통해 리눅스 시스템에서 프로세스의 동작 원리와 시그널 전송 및 처리 방식에 대해 깊이 있게 학습하는 것을 목적으로 한다.

2. 개발 범위 및 내용

A. 개발 범위

- 아래 항목을 구현했을 때의 결과를 간략히 서술

1. Phase 1

기본적인 shell 명령어 (예: ls, mkdir, cat) 실행이 가능하며, cd, quit와 같은 내장 명령어도 처리 가능하다.

2. Phase 2

'|' 연산자를 통해 명령어를 연결하여 여러 명령어들의 출력과 입력을 순차적으로 처리할 수 있도록 파이프라인을 구현하였다.

3. Phase 3

'&'를 명령어의 마지막에 붙임으로써 백그라운드에서 명령어를 실행할 수 있으며, Job 테이블을 구성하여 실행 중인 프로세스들을 추적하고 관리할 수 있게 하였다. 백그라운드에서 실행 중인 프로세스가 종료되었을 때는 SIGCHLD 시그널을 통해 부모 프로세스가 이를 reaping할 수 있도록 시그널 핸들러를 구현하였다. 또한 SIGINT, SIGCHLD, SIGTSTP, SIGCONT 시그널을 적절히 처리할 수 있는 시그널 핸들러도 등록하였다.

B. 개발 내용

- 아래 항목의 내용만 서술
- (기타 내용은 서술하지 않아도 됨. 코드 복사 붙여 넣기 금지)
- Phase1 (fork & signal)

✓ fork를 통해서 child process를 생성하는 부분에 대해서 설명

Fork() 시스템 호출을 통해 자식 프로세스를 생성하고, 자식 프로세스에서는 execve()를 통해 명령어 실행을 시도한다. 사용자가 입력한 명령어가 /로 시작하지 않는 경우에는 자동으로 /bin/ 경로를 붙여 실행 파일 경로를 완성한다.

부모 프로세스는 백그라운드 실행 여부를 판단한 후, 포그라운드 명령일 경우 waitpid()를 호출하여 자식 프로세스가 종료될 때까지 대기한다. 이 때 시그널 핸들러는 별도로 등록하지 않으며, 기본적인 동기적 wait 방식으로 프로세스를 수거한다.

내부 명령어(cd, quit)는 builtin_command() 함수에서 처리하며, 이 경우에는 별도의 자식 프로세스를 생성하지 않고 부모 프로세스에서 직접 실행된다.

✓ connection을 종료할 때 parent process에게 signal을 보내는 signal handling 하는 방법 & flow

이 프로그램에서 connection 종료 시 parent process 에게 signal 을 보내고 처리하는 방식은 SIGCHLD 시그널을 기반으로 동작한다. 자식 프로세스가 종료되면 커널은 자동으로 해당 부모에게 SIGCHLD 시그널을 전달하고, main 함수에서 등록된 시그널 핸들러 sigchld_handler 가 이를 처리한다. 핸들러는 waitpid(-1, &status, WNOHANG)을 통해 종료된 자식을 비동기적으로 reap 하며, 그 자식의 PID 를 전역 변수 pid_atomic 에 저장한다. 부모 프로세스는 자식이 foreground 로 실행 중인 경우에만 sigsuspend()를 통해 SIGCHLD 시그널을 기다리며 block 상태에 들어간다. 이때 부모는 pid_atomic 에 저장된 PID 가 자신이 fork 한 자식과 일치할 때까지 루프를 돌며 시그널 수신을 기다린다. 반면 background 로 실행된 자식 프로세스는 부모가 기다리지 않으며, 시그널 핸들러가 자동으로 reap 만 수행하고 pid_atomic 은 사용되지 않는다. 이를 통해 foreground 자식에 대해서는 명시적으로 종료를 기다리고, background 자식에 대해서는 자동으로 자원을 회수하여 좀비 프로세스를 방지하는 처리가 이루어진다.

- Phase2 (pipelining)

✓ Pipeline('|')을 구현한 부분에 대해서 간략히 설명 (design & implementation)

◆ **Design:** 사용자가 입력한 명령어를 | 연산자를 기준으로 분할하여 각 명령을 별도의 프로세스로 실행하도록 설계하였다. 각 명령어는 서로 연결된 파이프를 통해 출력과 입력을 주고받는다. 입력받은 명령어를 `insert_spaces_around_operators()`로 정리하고, 그 후 명령어 분할 및 파이프 연결은 `parseline()`과 `eval()` 함수에서 처리된다.

◆ Implementation:

- `parseline()` 함수에서 | 연산자를 기준으로 입력을 나누어 각 명령어와 인자를 `cmds[MAXCMDS][MAXARGS]` 배열에 저장한다.
- `eval()` 함수에서 명령어 수(`num_cmds`)만큼 반복하여 각 명령어를 `fork()`로 실행한다.
- 각 명령 사이에는 `pipe()`로 파이프를 생성하고, `dup2()`를 사용하여 이전 명령의 출력을 현재 명령의 입력으로 연결한다.
- 각 자식은 자신의 입력/출력을 파이프에 연결한 후, `execve()`를 통해 실제 명령어를 실행한다.
- 마지막 명령 이후에는 부모가 모든 파이프를 닫고, foreground job이면 `waitfg()`로 대기한다.

✓ Pipeline 개수에 따라 어떻게 handling했는지에 대한 설명

◆ 명령어가 1개인 경우: 일반적인 단일 명령 처리와 동일하게 `fork()`로 자식 생성 후 `execve()`를 통해 실행한다. 파이프는 생성하지 않는다.

◆ 명령어가 2개 이상인 경우:

- 명령 개수(`num_cmds`)에 따라 반복문을 수행하며 각 명령 사이에 `pipe()`를 생성한다.
- 반복문에서 `i < num_cmds - 1` 조건을 사용하여 마지막 명령이 아닌 경우에만 파이프를 생성하고 연결한다.

- 첫 번째 명령은 표준 입력을 그대로 사용하고, 출력만 파이프의 쓰기 끝으로 연결한다.
- 중간 명령들은 이전 파이프의 읽기 끝을 입력으로, 새 파이프의 쓰기 끝을 출력으로 연결한다.
- 마지막 명령은 이전 파이프의 읽기 끝을 입력으로만 사용하고, 출력은 표준 출력으로 둔다.
- 반복문에서 oldfd를 통해 이전 명령어의 파이프를 추적하여 닫을 타이밍도 조절한다.

- Phase3 (background process)

- ✓ Background ('&') process를 구현한 부분에 대해서 간략히 설명

◆ Design (설계):

- 사용자가 &를 명령어 끝에 입력하면, 해당 명령어를 **백그라운드 프로세스**로 실행하도록 설계하였다.
- 백그라운드로 실행되는 프로세스는 셸이 wait()하지 않고, 즉시 다음 입력을 받을 수 있게 한다.
- 이와 동시에, 해당 프로세스의 상태를 추적하기 위해 **Job 테이블**을 도입하여 별도로 관리한다.
- 각 백그라운드 프로세스는 고유의 **프로세스 그룹(process group)** 으로 묶고, Job 구조체를 통해 PID, PGID, JID, 상태 정보 등을 저장한다.

◆ Implementation (구현):

- parseline() 함수에서 명령어 끝의 '&' 토큰을 검사하여 bg 플래그를 BG로 설정하고, '&' 토큰을 명령어 목록에서 제거한다.
- eval() 함수에서 명령 실행 전, bg 여부를 확인한 후 fork()로 자식 프로세스를 생성하고 다음을 수행한다:
 - 자식 프로세스는 setpgid(0, 0)을 호출하여 자신을 새로운 프로세스 그룹의 리더로 설정한다.

- 이로 인해 포그라운드/백그라운드 job을 process group 단위로 제어할 수 있게 된다.
- `execve()`를 호출하여 명령어를 실제 실행하며, 실패 시에는 에러 메시지를 출력하고 종료한다.
- 부모 프로세스는 다음 흐름을 따른다:
 - 가장 먼저 생성한 자식의 PID를 `first_pid`로 저장하고, 이후 모든 자식 프로세스를 해당 PID를 가진 프로세스 그룹으로 설정한다 (`setpgid(pid, first_pid)`).
 - 시그널 레이스를 방지하기 위해 `SIGCHLD`를 block한 상태에서 `addjob()`을 호출해 **Job 테이블에 현재 명령어를 등록**한다.
 - 이후 시그널 마스크를 다시 복원하여 unblock 상태로 되돌린다.
 - 만약 `bg == BG`이면, `getjobpid()`로 job 정보를 가져와 `[jid] pgid` 형식으로 백그라운드 실행 정보를 출력한다.
- 이후 `jobs`, `fg`, `bg`, `kill` 등의 내장 명령어에서 이 Job 테이블을 참조하여 제어할 수 있게 된다

C. 개발 방법

- B.의 개발 내용을 구현하기 위해 어느 소스코드에 어떤 요소를 추가 또는 수정할 것인지 설명. (함수, 구조체 등의 구현이나 수정을 서술)

- Phase1 (fork & signal)

◆ `eval()` 함수 구현

- 사용자가 입력한 명령어를 처리하기 위한 핵심 함수로 직접 구현함.
- `fork()`를 사용하여 자식 프로세스를 생성하고, 자식에서는 `execve()`를 호출하여 외부 명령어 실행.
- `/bin/` 경로를 명시하지 않은 명령어의 경우 자동으로 접두 경로(`/bin/`)를 붙여서 실행 파일을 찾도록 구현.
- 부모 프로세스는 자식 프로세스가 foreground job일 경우 `waitpid()`를 통해

종료될 때까지 대기.

◆ **parseline() 함수 구현**

- 사용자 입력을 공백 기준으로 토큰화하여 argv 배열을 구성.
- 입력 끝에 &가 있을 경우, 이를 제거하고 bg 플래그를 설정하여 백그라운드 실행 여부를 반환.

◆ **builtin_command() 함수 구현**

- cd, quit, & 명령어를 처리할 수 있도록 구현.
- cd 명령어는 인자가 없을 경우 홈 디렉토리로 이동하며, 인자가 있는 경우 해당 디렉토리로 이동하도록 chdir() 사용.
- quit은 즉시 셸을 종료하도록 exit(0) 호출.
- 단독 &는 무시하도록 함 (return 1).

◆ 기타 요소

- main() 함수에서는 사용자로부터 명령어를 반복적으로 입력받고, eval()을 호출하여 처리하는 루프를 구성.
- feof()를 통해 EOF(Ctrl+D)를 감지하고 셸 종료 처리.

- **Phase2 (pipelining)**

◆ **eval() 함수 수정**

- 기존 단일 명령 실행 로직을 확장하여, 여러 개의 명령어가 '|' 연산자를 기준으로 연결된 경우 **반복문을 통해 각 명령어를 별도의 프로세스로 실행하도록** 구현함.
- 각 명령어 간 **pipe()**를 사용하여 **파이프라인을 구성**하고, dup2()를 통해 이전 명령어의 출력을 다음 명령어의 입력으로 전달함.
- 각 자식 프로세스는 자신에게 할당된 입력(stdin)과 출력(stdout)을 적절히 설정한 후 execve()로 실제 명령어를 실행함.
- 부모 프로세스는 파이프의 불필요한 파일 디스크립터를 닫아 자식과의 입출

력을 마무리하고, foreground 실행 시에는 wait() 루프를 통해 모든 자식 프로세스의 종료를 기다림.

◆ **parseline() 함수 수정**

- 명령어 파싱 시 '|' 연산자를 기준으로 명령어를 분리하여 **cmds[MAXCMDS][MAXARGS]** 배열에 저장하는 구조로 변경.
- 이를 위해 공백을 기준으로 자르고, '|' 기호를 발견할 경우 새로운 명령어 그룹으로 전환되도록 구현함.
- &가 명령어 끝에 있을 경우 bg 플래그를 설정하여 백그라운드 실행 여부를 함께 전달함.

◆ **insert_spaces_around_operators() 함수 추가**

- "ls|grep foo"처럼 연산자 주변에 공백이 없는 경우에도 정확하게 파싱할 수 있도록 하기 위해, '|', '<', '>', '&' 연산자 주변에 자동으로 공백을 삽입함.
- parseline()에서 가공된 입력 문자열은 항상 이 전처리 과정을 거치도록 구현함.

◆ **기타 보조 함수**

- **strip_quotes() 함수 추가:** 인자 중에 "따옴표"가 포함된 경우 이를 제거하여 명령어 인자들이 제대로 인식되도록 처리함.

Ex> grep -i "abc"와 grep -i abc는 같은 명령어.

- **Phase3 (background process)**

◆ **fg_done과 running 전역 변수 사용**

- running은 pipeline 구조에서 fork된 자식 프로세스의 개수를 저장하는 전역 변수로, eval() 함수에서 명령어 수(num_cmds)만큼 설정됨.
- sigchld_handler()에서 자식 프로세스가 종료되거나 중단될 때마다 running--을 수행하며, 모든 자식이 종료되었을 경우 fg_done = 1로 설정함.
- waitfg() 함수는 fg_done이 1이 될 때까지 sigsuspend()를 통해 기다리며, 이를 통해 foreground job이 완전히 종료될 때까지 셸이 블록되도록 구현함.
- 이 두 변수는 특히 다중 파이프 명령어에서 여러 자식의 종료 시점을 정확히

감지하고, signal 기반 비동기 처리에서 정확한 동기화를 가능하게 함.

◆ job_t 구조체 및 전역 배열 정의

- job_t 구조체를 새로 정의하여 각 job에 대한 정보를 저장함.

```
typedef struct job_t {  
    pid_t pid;           // job의 프로세스 ID (보통 첫 번째)  
    pid_t pgid;          // 프로세스 그룹 ID (= job 전체를 대표)  
    int jid;             // job ID (1부터 시작)  
    job_state state;     // job의 상태: UNDEF, FG, BG, ST  
    char cmdline[MAXLINE]; // 사용자가 입력한 명령 전체  
} job_t;
```

- 이 구조체는 전역 배열 jobs[MAXJOBS]로 선언되어 여러 개의 job을 동시에 추적 및 관리할 수 있도록 함.
- nextjid 변수를 통해 새로운 job ID를 자동 부여하도록 설정함.

◆ job table 관리 함수 추가

- addjob(), deletejob(), getjobpid(), getjobjid(), listjobs() 등 job을 추가/삭제/검색/출력하는 함수들을 직접 구현함.
- waitfg()는 포그라운드 job이 종료되기 전까지 셸이 블록되도록 구현함.
- fgpuid()는 현재 포그라운드 job의 프로세스 그룹 ID를 반환함.

◆ builtin 명령어 처리 확장

- 기존 builtin_command() 함수에 다음 명령어들을 추가함:
 - jobs: 현재 실행 중이거나 중지된 job 리스트 출력
 - fg %jid: 지정된 job을 포그라운드로 전환 후 waitfg() 호출
 - bg %jid: 지정된 job을 백그라운드로 전환하고 SIGCONT 시그널 전송
 - kill %jid or pid: 지정된 job 또는 프로세스를 종료시키기 위해 SIGTERM 전송

◆ background 실행을 위한 프로세스 그룹 설정

- eval() 내에서 첫 번째 자식의 PID를 프로세스 그룹 리더(PGID)로 설정하고, 이후 자식들은 setpgid(pid, first_pid)를 호출하여 동일한 그룹으로 설정함.
- background 실행 시 addjob()을 통해 job 리스트에 등록하고, [jid] pgid 형식

으로 출력함.

◆ 시그널 핸들러 구현

- sigchld_handler: 자식 프로세스가 종료되거나 중단될 때 호출되어 job 상태를 갱신하고, 자식 PID를 기반으로 job을 삭제함.
- sigint_handler: 사용자가 Ctrl+C를 눌렀을 때, 현재 foreground job의 PGID에 SIGINT 시그널을 전달하여 종료시킴.
- sigtstp_handler: 사용자가 Ctrl+Z를 눌렀을 때, foreground job에 SIGTSTP 시그널을 전달하여 중지시킴.

◆ 시그널 마스크 처리

- eval()에서 race condition을 방지하기 위해 sigset_t을 사용하여 SIGCHLD를 블록한 상태에서 addjob()을 호출하고, 이후 unblock 처리함.

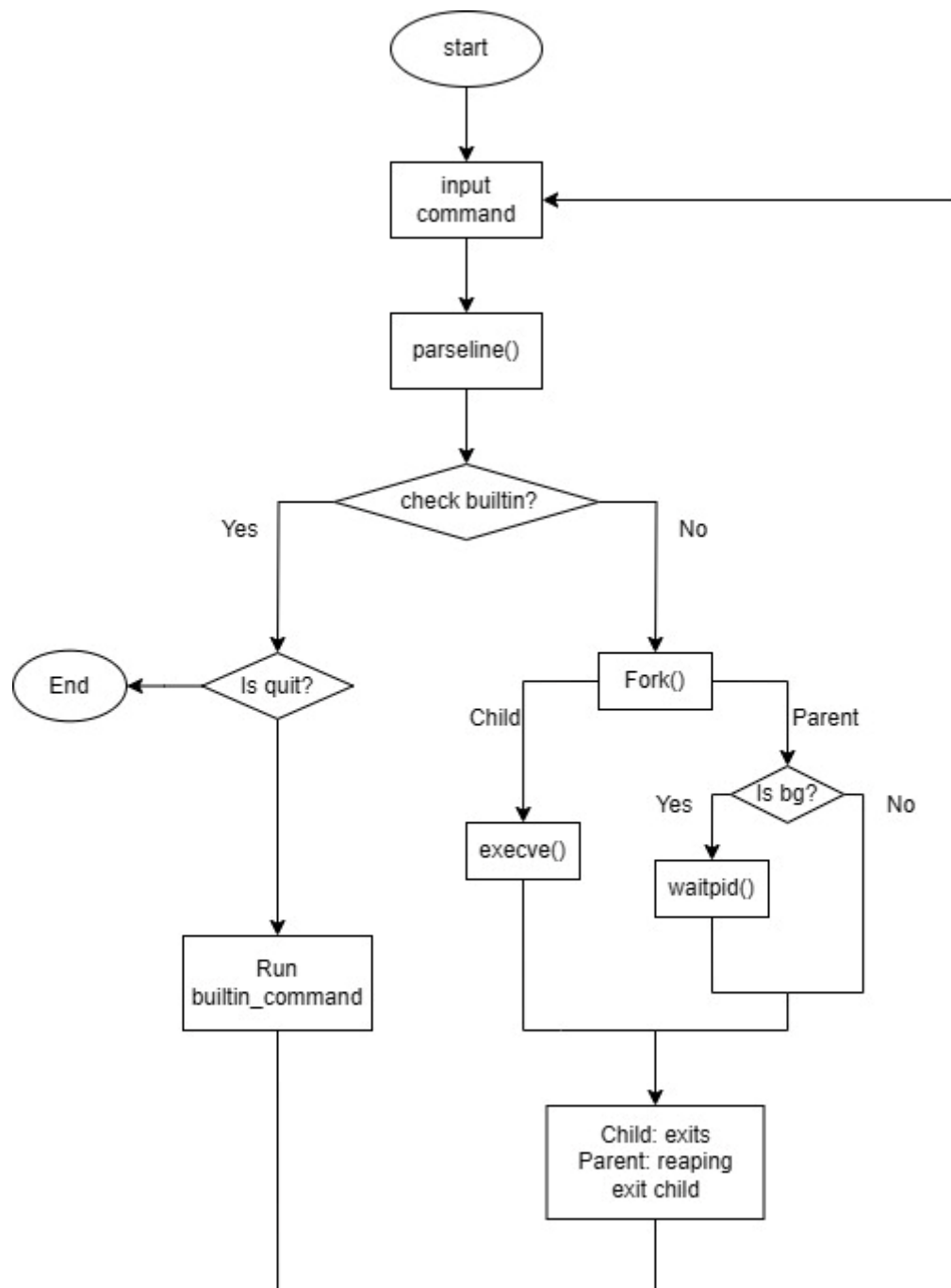
3. 구현 결과

A. Flow Chart

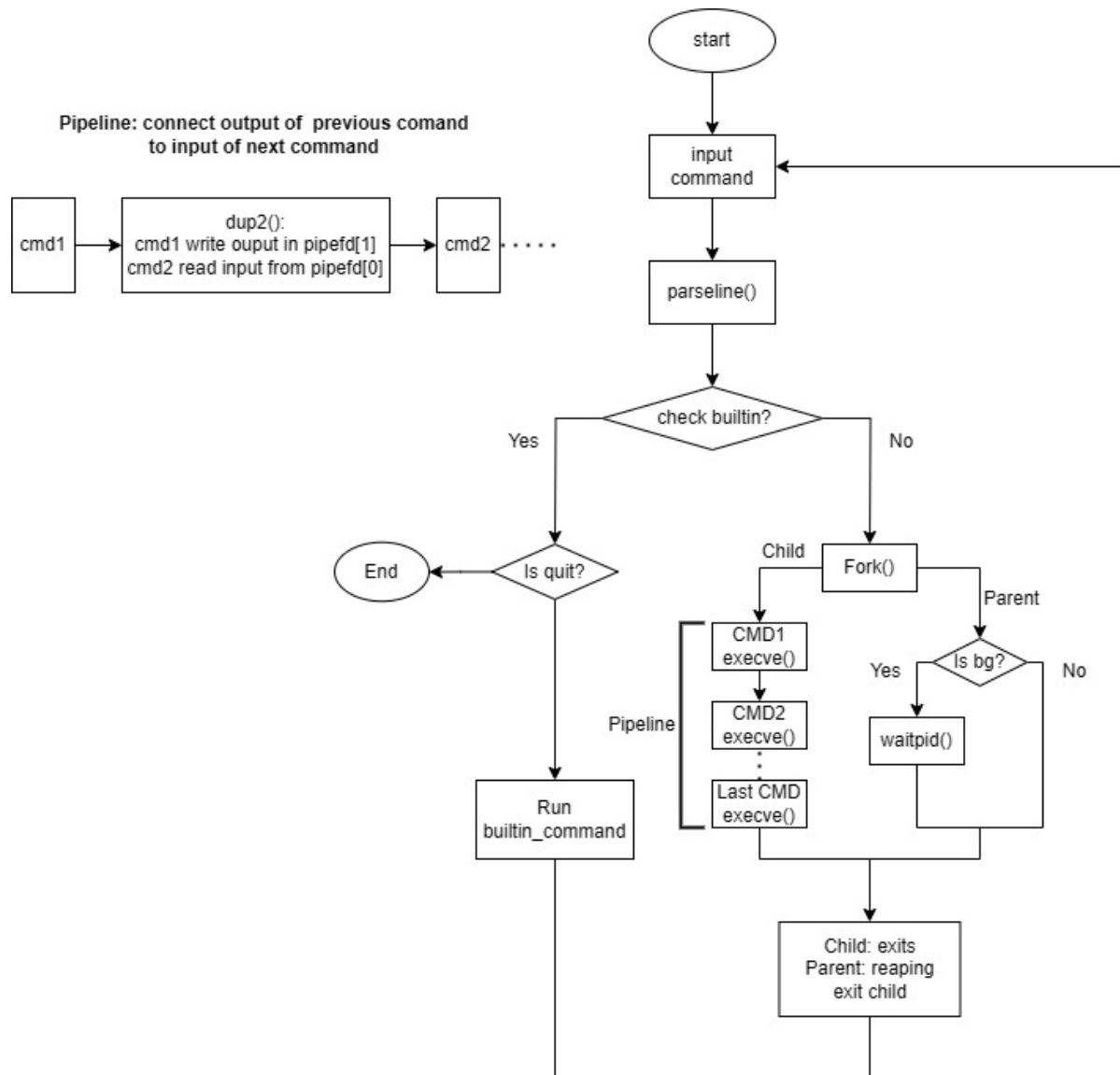
- 2.B.개발 내용에 대한 Flow Chart를 작성.
- (각각의 방법들에서 추가된 내용(fork, pipeline, background)만 특성이 잘 드러나게 그리면 됨.)

(다음 페이지)

Phase 1 (fork)



Phase 2 (pipeline)



Phase 3 (background)

